



INE-PYTHON

LECTURE - 11 PYTHON OOP

What You Will Learn

- Introduction
- Class and Object
- Attribute and Method
- Inheritance
- Encapsulation
- Abstraction
- Polymorphism

OOP



Introduction

OBJECT ORIENTED PROGRAMMING

การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) เป็นแนวทางการเขียนโปรแกรมที่ทรงพลัง โดยมี "อ็อบเจกต์" เป็นแกนหลัก ซึ่งอ็อบเจกต์คือตัวอย่าง (instance) ของคลาส ที่รวมข้อมูลไว้ในรูปของแอตทริบิวต์ และพฤติกรรมในรูปของเมธอด OOP ถูกออกแบบมาเพื่อช่วยจัดระเบียบโค้ด เพิ่มความสามารถในการนำกลับมาใช้ และขยายระบบได้อย่างมีประสิทธิภาพ

หลักการสำคัญของ OOP ได้แก่ การห่อหุ้ม (encapsulation) ซึ่งเป็นการรวมข้อมูลและเมธอดไว้ในหน่วยเดียว การสืบทอด (inheritance) ซึ่งทำให้สามารถสร้างคลาสใหม่โดยใช้คุณสมบัติของคลาสเดิม และพอลิมอร์ฟิซึม (polymorphism) ซึ่งทำให้เมธอดสามารถทำงานต่างกันตามอ็อบเจกต์ที่ใช้งาน

OOP มอบโครงสร้างที่ชัดเจนโดยการจัดโปรแกรมให้เป็นอ็อบเจกต์และคลาส ช่วยให้จัดการระบบที่ซับซ้อนได้ง่ายขึ้น และเชื่อมโยงแนวคิดในโลกจริงเข้ากับโค้ดได้อย่างเป็นธรรมชาติ บทนี้จะเจาะลึกแนวคิดเหล่านี้พร้อมตัวอย่างการใช้งานใน Python

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING IN PYTHON

การเขียนโปรแกรมเชิงวัตถุ (OOP)

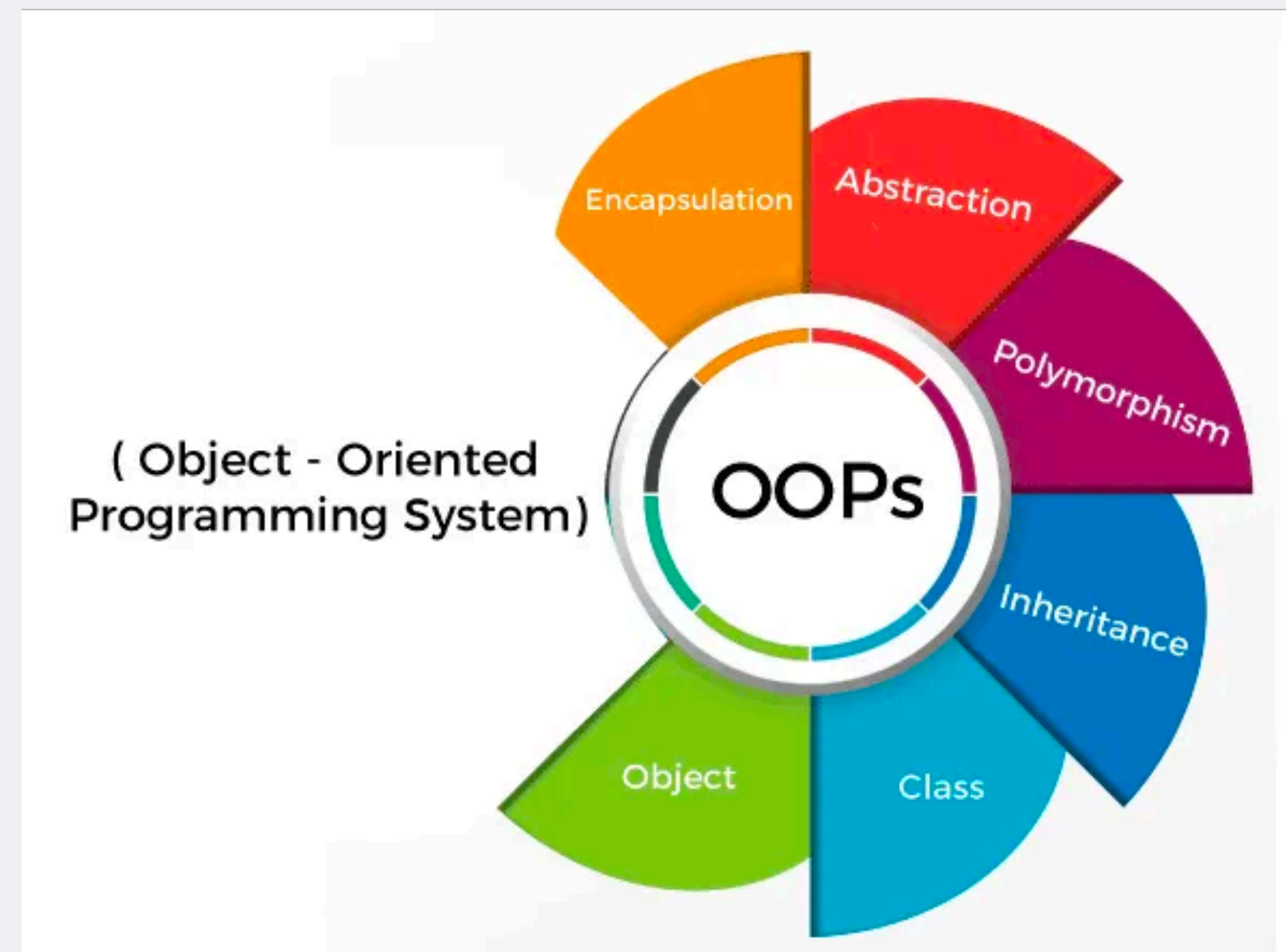
- วัตถุ (Objects): อินสแตนซ์ของคลาสที่มีข้อมูลและเมธอดในตัว
- คลาส (Classes): แม่แบบสำหรับสร้างวัตถุ โดยกำหนดโครงสร้างและพฤติกรรม
- การห่อหุ้ม (Encapsulation): รวมข้อมูลและเมธอดในหน่วยเดียว และจำกัดการเข้าถึง
- การสืบทอด (Inheritance): คลาสใหม่สามารถสืบทอดคุณสมบัติและเมธอดจากคลาสเดิม
- พอลิมอร์ฟิซึม (Polymorphism): เมธอดสามารถทำงานต่างกันตามวัตถุที่ใช้งาน
- นามธรรม (Abstraction): ซ่อนรายละเอียดที่ซับซ้อน และเปิดเผยเฉพาะสิ่งจำเป็นให้ผู้ใช้เห็น
- ความเป็นโมดูล (Modularity): แยกโค้ดเป็นส่วน ๆ ช่วยให้เข้าใจและจัดการได้ง่าย
- การนำกลับมาใช้ (Reusability): เขียนโค้ดให้สามารถนำไปใช้ซ้ำได้ในหลายส่วนหรืองานอื่น

OBJECT-ORIENTED AND PROCEDURAL ORIENTED PROGRAMMING

คุณสมบัติ	OOP (การเขียนโปรแกรมเชิงวัตถุ)	POP (การเขียนโปรแกรมเชิงโปรซีเยอร์)
แนวคิดหลัก	อิงกับอ็อบเจกต์และคลาส	อิงกับฟังก์ชันและกระบวนการ
จุดเน้นหลัก	เน้นอ็อบเจกต์ที่รวมข้อมูลและเมธอดไว้ด้วยกัน	เน้นลำดับขั้นตอนการทำงาน (procedures)
การเข้าถึงข้อมูล	จำกัดการเข้าถึงผ่านเมธอด (encapsulation)	ฟังก์ชันสามารถเข้าถึงและเปลี่ยนข้อมูลได้โดยตรง
ความเป็นโมดูล	โค้ดแบ่งเป็นอ็อบเจกต์ (อินสแตนซ์ของคลาส)	โค้ดแบ่งเป็นฟังก์ชันหรือขั้นตอน
การนำกลับมาใช้	มีสูง: ผ่านการสืบทอด พอลิมอร์ฟิซึม และคลาส	มีน้อย: มักต้องคัดลอกโค้ดซ้ำ
ความปลอดภัยของข้อมูล	สูง: มีการป้องกันข้อมูลด้วยการห่อหุ้ม	ต่ำ: ข้อมูลเปิดให้ทุกฟังก์ชันเข้าถึงได้
การดูแลรักษาโค้ด	ง่าย: โครงสร้างเป็นโมดูล ปรับปรุงได้เป็นส่วน ๆ	ยาก: การเปลี่ยนแปลงอาจกระทบหลายส่วน
ภาษาโปรแกรมตัวอย่าง	Python, Java, C++, Ruby, C#	C, Pascal, FORTRAN, Basic
การทำงาน	เน้นการโต้ตอบระหว่างอ็อบเจกต์	ทำงานเป็นลำดับจากบนลงล่าง
การสืบทอด	รองรับ: คลาสสามารถสืบทอดกันได้	ไม่รองรับการสืบทอด
พอลิมอร์ฟิซึม	รองรับ: เมธอดเดียวกันใช้กับอ็อบเจกต์ต่างกันได้	ไม่รองรับพอลิมอร์ฟิซึม
นามธรรม	รองรับ: ซ่อนความซับซ้อนจากผู้ใช้	ไม่มีแนวคิดเรื่องนามธรรม
แนวทางการพัฒนา	จากล่างขึ้นบน (Bottom-up)	จากบนลงล่าง (Top-down)

Table 11.1: เปรียบเทียบระหว่างการเขียนโปรแกรมเชิงวัตถุ (OOP) และเชิงโปรซีเยอร์ (POP)

OOP METHODOLOGIES



- Inheritance: child classes inherit data and behaviors from the parent class
- Encapsulation: containing information in an object, exposing only selected information
- Abstraction: only exposing high-level public methods for accessing an object
- Polymorphism: many methods can do the same task

OOP



Class and Object

CLASSES AND OBJECTS

คลาสและอ็อบเจกต์เป็นพื้นฐานสำคัญของการเขียนโปรแกรมเชิงวัตถุ (OOP) การเข้าใจวิธีใช้อย่างมีประสิทธิภาพมีความสำคัญอย่างยิ่งในการเขียนโค้ดให้สะอาด เป็นโมดูล และนำกลับมาใช้ได้

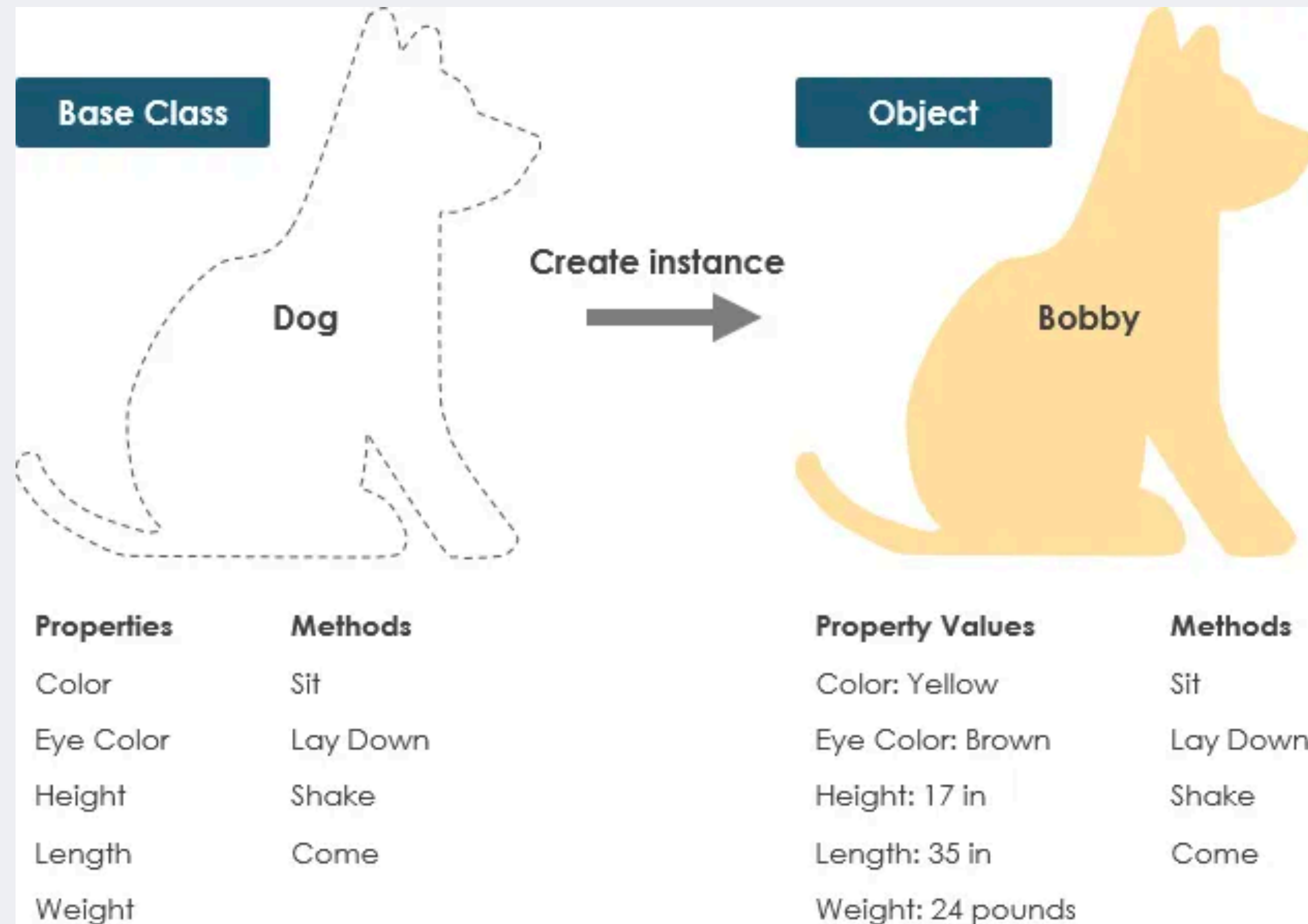
คลาส (Classes)

คลาสคือแบบพิมพ์เขียวสำหรับการสร้างอ็อบเจกต์ โดยกำหนดชุดของคุณลักษณะ (ข้อมูล) และเมธอด (ฟังก์ชัน) ที่อ็อบเจกต์ซึ่งสร้างจากคลาสนั้นสามารถใช้งานได้ ให้นึกว่าคลาสคือแม่แบบที่ระบุว่าอ็อบเจกต์จะมีลักษณะและพฤติกรรมอย่างไร

ไวยากรณ์:

```
1 class ClassName:  
2     # Class attributes  
3     # Class methods
```


CLASSES AND OBJECTS



CLASSES AND OBJECTS

```
1 class Car:
2     # Class attribute
3     wheels = 4
4
5     # Initializer method (constructor)
6     def __init__(self, make, model, year):
7         self.make = make # Instance attribute
8         self.model = model # Instance attribute
9         self.year = year # Instance attribute
10
11     # Method
12     def start_engine(self):
13         return f"The engine of the {self.year} {self.make} {self.model} is
14             now running."
15
16     # Method
17     def stop_engine(self):
18         return f"The engine of the {self.year} {self.make} {self.model} is
19             now off."
```

Listing 11.1: Example of a Class

CLASSES AND OBJECTS

```
1 # Creating an instance of the Car class
2 my_car = Car("Toyota", "Corolla", 2020)
```

Listing 11.2: Creating an Object

```
1 # Accessing instance attributes
2 print(my_car.make)    # Output: Toyota
3 print(my_car.model)   # Output: Corolla
4 print(my_car.year)    # Output: 2020
5
6 # Calling instance methods
7 print(my_car.start_engine()) # Output: The engine of the 2020 Toyota Corolla is
    now running.
8 print(my_car.stop_engine()) # Output: The engine of the 2020 Toyota Corolla is now
    off.
```

Listing 11.3: Accessing Attributes and Methods

CLASSES AND OBJECTS

```
1 class Book:
2     def __init__(self, title, author, isbn):
3         self.title = title
4         self.author = author
5         self.isbn = isbn
6         self.is_checked_out = False
7
8     def check_out(self):
9         if not self.is_checked_out:
10             self.is_checked_out = True
11             return f"'{self.title}' has been checked out."
12         else:
13             return f"'{self.title}' is already checked out."
14
15     def return_book(self):
16         if self.is_checked_out:
17             self.is_checked_out = False
18             return f"'{self.title}' has been returned."
19         else:
20             return f"'{self.title}' was not checked out."
21
22 # Creating book objects
23 book1 = Book("1984", "George Orwell", "1234567890")
24 book2 = Book("To Kill a Mockingbird", "Harper Lee", "0987654321")
25
26 # Checking out and returning books
27 print(book1.check_out()) # Output: '1984' has been checked out.
28 print(book1.return_book()) # Output: '1984' has been returned.
29 print(book2.check_out()) # Output: 'To Kill a Mockingbird' has been checked out.
```

Listing 11.4: Example of a Practical Application

BENEFITS OF USING CLASSES AND OBJECTS

ข้อดีของการใช้คลาสและอ็อบเจกต์

- **ความเป็นโมดูล (Modularity):** คลาสช่วยให้สามารถแบ่งปัญหาที่ซับซ้อนออกเป็นส่วนย่อยที่จัดการได้ง่าย
- **การนำกลับมาใช้ (Reusability):** เมื่อกำหนดคลาสไว้แล้ว สามารถนำไปสร้างอ็อบเจกต์ได้หลายตัว ช่วยลดความซ้ำซ้อน
- **การดูแลรักษา (Maintainability):** การเปลี่ยนแปลงภายในคลาสจะมีผลกับอ็อบเจกต์ทั้งหมด ทำให้การปรับปรุงระบบง่ายขึ้น
- **การห่อหุ้ม (Encapsulation):** คลาสช่วยรวมข้อมูลและฟังก์ชันไว้ด้วยกัน พร้อมจำกัดการเข้าถึงบางส่วน เพื่อปกป้องความถูกต้องของข้อมูล

OOP



Attribute

ATTRIBUTE

OOP >  Ex1x1-ClassInstanceAtt.py > ...

```
1  class Dog:
2
3      # Class Attribute
4      species = 'mammal'
5
6      # Initializer / Instance Attributes
7      def __init__(self, name, age):
8          self.name = name
9          self.age = age
10
11     # Instantiate the Dog object
12     dog1 = Dog("Philo", 5)
13     dog2 = Dog("Mikey", 6)
14
15     # Access the instance attributes
16     print("{} is {} and {} is {}."
17           .format(dog1.name, dog1.age, dog2.name, dog2.age))
18
19     # Is Philo a mammal?
20     if dog1.species == "mammal":
21         print("{} is a {}!".format(dog1.name, dog1.species))
```

species is Class Attribute

name, age is Instance Attribute

__init__ is C

```
Philo is 5 and Mikey is 6.
Philo is a mammal!
```

OOP



Methods

METHODS

OOP >  Ex1x2-InstanceMethod.py > ...

```
1  class Dog:
2
3      # Class Attribute
4      species = 'mammal'
5
6      # Initializer / Instance Attributes
7      def __init__(self, name, age):
8          self.name = name
9          self.age = age
10
11     # instance method
12     def description(self):
13         return "{} is {} years old".format(self.name, self.age)
14
15     # instance method
16     def speak(self, sound):
17         return "{} says {}".format(self.name, sound)
18
19     # Instantiate the Dog object
20     mikey = Dog("Mikey", 6)
21
22     # call our instance methods
23     print(mikey.description())
24     print(mikey.speak("Gruff Gruff"))
```

Class Instance Methods

```
Mikey is 6 years old
Mikey says Gruff Gruff
```


OOP >  Ex1x3-Methods.py > ...

```
1  class Calculate_area:
2
3      # Instance Method
4      def rectangle_area(self, w, h):
5          return w * h
6
7      @classmethod
8      def triangle_area(cls, b, h):
9          return 0.5 * b * h
10
11     @staticmethod
12     def circle_area(r):
13         return 3.14 * r * r
14
15     cal = Calculate_area()
16     cal_rec = cal.rectangle_area(4, 5)
17     cal_tri = cal.triangle_area(4, 5)
18     cal_circle = cal.circle_area(5)
19
20     print('Rectangle Area = ', cal_rec)
21     print('Triangle Area = ', cal_tri)
22     print('Circle Area = ', cal_circle)
23
24     #print('Test Triangle Area', Calculate_area.triangle_area(5, 6))
25     #print('Test Circle Area', Calculate_area.circle_area(5))
26     #print('Test Rectangle Area', Calculate_area.rectangle_area(5, 6))
```

METHODS

Instance Method

Class Method

Static Method

```
Rectangle Area = 20
Triangle Area = 10.0
Circle Area = 78.5
```

METHODS

OOP >  Ex1x4-STRMethod.py > ...

```
1  class StudentTest:
2      def __init__(self, name, score1, score2, score3):
3          self.name = name
4          self.score1 = score1
5          self.score2 = score2
6          self.score3 = score3
7
8      def sumScore(self):
9          return self.score1 + self.score2 + self.score3
10
11     def __str__(self):
12         return 'Name:{}, Total of score: {}'.format(self.name, self.sumScore())
13
14     std1 = StudentTest('Jantra', 20, 35, 25)
15     print(std1.name, std1.sumScore())
16     print(std1)
```

`__str__`

Ex ..Special Methods

Jantra 80

Name:Jantra, Total of score: 80

OOP



Inheritance

INHERITANCE

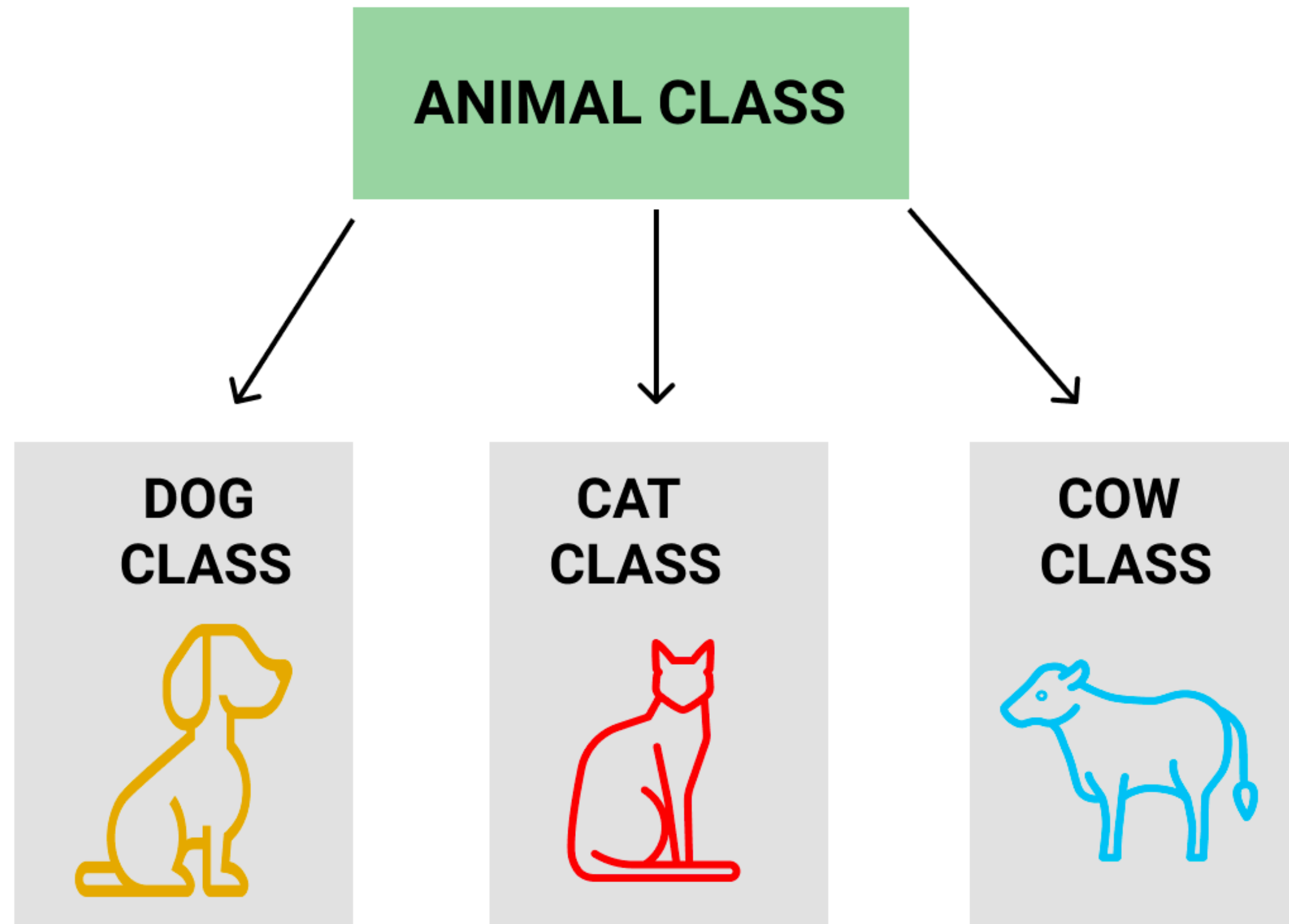
การสืบทอด (Inheritance) ช่วยให้คลาสสามารถรับคุณสมบัติและเมธอดจากคลาสอื่นได้ ซึ่งเป็นกลไกในการสร้างคลาสใหม่จากคลาสที่มีอยู่แล้ว คุณสมบัตินี้ช่วยให้สามารถนำโค้ดกลับมาใช้ใหม่ และสร้างโครงสร้างลำดับชั้น (hierarchical structure) ของคลาสอย่างเป็นธรรมชาติ

คลาสที่ถูกสืบทอดเรียกว่า คลาสแม่ หรือ คลาสฐาน (parent หรือ base class) ซึ่งมักจะประกอบด้วยคุณลักษณะและเมธอดทั่วไปที่ใช้ร่วมกันได้ ส่วนคลาสที่ทำการสืบทอดเรียกว่า คลาสลูก หรือ คลาสที่สืบทอดมา (child หรือ derived class) โดยคลาสลูกจะได้รับคุณสมบัติและพฤติกรรมจากคลาสแม่ และยังสามารถเพิ่มคุณสมบัติเฉพาะของตนเองหรือเขียนเมธอดใหม่เพื่อให้เหมาะสมกับการใช้งานเฉพาะทางได้อีกด้วย

โครงสร้างแบบลำดับชั้นนี้ทำให้นักพัฒนาสามารถสร้างระบบที่ซับซ้อนได้ด้วยโครงสร้างที่ชัดเจน โดยมีฟังก์ชันหลักอยู่ในคลาสฐาน และฟีเจอร์เฉพาะอยู่ในคลาสลูก ส่งผลให้โค้ดสะอาด มีระเบียบ และดูแลรักษาง่าย

```
1 class ParentClass:
2     # Parent class attributes and methods
3
4 class ChildClass(ParentClass):
5     # Child class attributes and methods
```

INHERITANCE



INHERITANCE

```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def speak(self):
6         return "Some sound"
7
8 class Dog(Animal):
9     def speak(self):
10        return f"{self.name} says woof!"
11
12 class Cat(Animal):
13     def speak(self):
14        return f"{self.name} says meow!"
15
16 # Creating objects of child classes
17 dog = Dog("Buddy")
18 cat = Cat("Whiskers")
19
20 print(dog.speak()) # Output: Buddy says woof!
21 print(cat.speak()) # Output: Whiskers says meow!
```

Listing 11.5: Example of Inheritance

INHERITANCE

```
OOP >  Ex2x2-OverridingFunction.py > ...  
1  class Dog:  
2      species = 'mammal'  
3  
4      def calAge(self,age):  
5          print('Dog Age is {}'.format(age*3))  
6  
7  class SomeBread(Dog):  
8      pass  
9  
10 class SomeOtherBread(Dog):  
11     species = 'reptile'  
12     def calAge(self,age):  
13         print('Dog Age is {}'.format(age*4))  
14  
15 frank = SomeBread()  
16 print(frank.species)  
17 frank.calAge(5)  
18  
19 beans = SomeOtherBread()  
20 print(beans.species)  
21 beans.calAge(5)
```

```
mammal  
Dog Age is 15  
reptile  
Dog Age is 20
```

OOP



Encapsulation

ENCAPSULATION

การห่อหุ้ม (Encapsulation) เป็นแนวคิดพื้นฐานใน OOP ที่รวมข้อมูล (แอตทริบิวต์) และเมธอด (ฟังก์ชัน) ที่ทำงานกับข้อมูลนั้นไว้ในหน่วยเดียวกัน โดยปกติคือภายในคลาส การห่อหุ้มจะจำกัดการเข้าถึงองค์ประกอบภายในบางส่วน ของอ็อบเจกต์ ซึ่งช่วยป้องกันการเปลี่ยนแปลงหรือการใช้งานข้อมูลโดยไม่ตั้งใจ และยังช่วยให้โค้ดเป็นโมดูลและดูแลรักษาได้ง่ายขึ้น

หลักการสำคัญของการห่อหุ้ม

- **แอตทริบิวต์และเมธอดแบบส่วนตัว (Private):** การกำหนดให้แอตทริบิวต์หรือเมธอดเป็นแบบส่วนตัว โดยใช้ขีดล่างสองตัว (__) หน้าชื่อ จะป้องกันไม่ให้อื่นสามารถเข้าถึงได้จากภายนอกคลาส
- **เมธอดแบบสาธารณะ (Public):** เมธอดประเภทนี้ เช่น getter และ setter ใช้ในการเข้าถึงและปรับเปลี่ยนค่าของแอตทริบิวต์ส่วนตัวในลักษณะที่ควบคุมได้
- **การซ่อนข้อมูล (Information Hiding):** การห่อหุ้มช่วยซ่อนรายละเอียดของการทำงานภายในอ็อบเจกต์ และเปิดเผยเฉพาะส่วนที่จำเป็นผ่านเมธอดสาธารณะ ทำให้ใช้งานง่ายขึ้นและลดความผิดพลาด

ENCAPSULATION

```
OOP >  Ex5-Encap1.py > ...
1  class employee(object):
2      def __init__(self):
3          self.name = 'Peter'
4          self._age = 45
5          self.__salary = 35000
6
7  object1 = employee()
8  print(object1.name)
9  print(object1._age)
10 print(object1.__salary)
11
```

You will get this question what is the underscore and error? Well, python class treats the private variables as(__salary) which can not be accessed directly.

```
Peter
```

```
45
```

```
Traceback (most recent call last):
```

```
File "/Users/anirachmingkhwan/Code/INE-Python/OOP/Ex5-Encap1.py", line 10, in <module>
```

```
    print(object1.__salary)
```

```
AttributeError: 'employee' object has no attribute '__salary'
```

ENCAPSULATION

OOP >  Ex6-Encap2.py > ...

```
1 class employee():
2     def __init__(self):
3         self.__maxearn = 30000
4
5     def earn(self):
6         print("earning is:{}".format(self.__maxearn))
7
8     # setter method used for accesing private class
9     def setmaxearn(self, earn):
10        self.__maxearn = earn
11
12 emp1 = employee()
13 emp1.earn()
14
15 emp1.__maxearn = 10000
16 emp1.earn()
17
18 emp1.setmaxearn(15000)
19 emp1.earn()
```

```
earning is:30000
earning is:30000
earning is:15000
```

Making Use of the setter method provides indirect access to the private class method. Here I have defined a class employee and used a (`__maxearn`) which is the setter method used here to store the maximum earning of the employee, and a setter function `setmaxearn()` which is taking price as the parameter.

This is a clear example of encapsulation where we are restricting the access to private class method and then use the setter method to grant access.

OOP



Polymorphism

POLYMORPHISM

พอลิมอร์ฟิซึม (Polymorphism) เป็นแนวคิดหลักใน OOP ซึ่งช่วยให้เมธอดสามารถทำงานแตกต่างกันตามประเภทของอ็อบเจกต์ที่ถูกเรียกใช้งาน แม้จะใช้ชื่อเมธอดเดียวกันก็ตาม คำว่า "Polymorphism" แปลว่า "หลายรูปแบบ" ซึ่งหมายถึงความสามารถของอ็อบเจกต์ที่แตกต่างกันในการตอบสนองต่อคำสั่งเดียวกันในลักษณะที่ต่างกัน

พอลิมอร์ฟิซึมช่วยเพิ่มความยืดหยุ่นและความสามารถในการขยายของโค้ด โดยเปิดทางให้สามารถใช้ส่วนติดต่อเดียวกัน (interface) กับอ็อบเจกต์หลากหลายชนิดที่มีพฤติกรรมต่างกัน

ประเภทหลักของพอลิมอร์ฟิซึม

- **การเขียนเมธอดซ้ำ (Method Overriding)** หรือพอลิมอร์ฟิซึมขณะรันไทม์ (Runtime Polymorphism): เกิดขึ้นเมื่อคลาสลูกเขียนเมธอดใหม่โดยใช้ชื่อเดียวกับในคลาสแม่ เพื่อกำหนดพฤติกรรมเฉพาะของตนเอง
- **การใช้เมธอดซ้ำชื่อ (Method Overloading)** หรือพอลิมอร์ฟิซึมขณะคอมไพล์ไทม์ (Compile-time Polymorphism): Python ไม่รองรับโดยตรง แต่สามารถเลียนแบบได้โดยกำหนดพารามิเตอร์ที่แตกต่างกันในเมธอด

POLYMORPHISM

```
1 class Animal:
2     def speak(self):
3         raise NotImplementedError("Subclass must implement abstract method")
4
5 class Dog(Animal):
6     def speak(self):
7         return "Woof!"
8
9 class Cat(Animal):
10    def speak(self):
11        return "Meow!"
12
13 # Polymorphism in action
14 def make_animal_speak(animal):
15     print(animal.speak())
16
17 dog = Dog()
18 cat = Cat()
19
20 make_animal_speak(dog)    # Output: Woof!
21 make_animal_speak(cat)    # Output: Meow!
```

Listing 11.8: Example of Method Overriding

POLYMORPHISM

```
1 class Shape:
2     def area(self):
3         raise NotImplementedError("Subclass must implement abstract method")
4
5 class Rectangle(Shape):
6     def __init__(self, width, height):
7         self.width = width
8         self.height = height
9
10    def area(self):
11        return self.width * self.height
12
13 class Circle(Shape):
14     def __init__(self, radius):
15         self.radius = radius
16
17    def area(self):
18        import math
19        return math.pi * (self.radius ** 2)
20
21 # List of different shapes
22 shapes = [Rectangle(10, 20), Circle(5)]
23
24 for shape in shapes:
25     print(f"Area: {shape.area()}")
26 # Output:
27 # Area: 200
28 # Area: 78.53981633974483
```

Listing 11.9: Example of Polymorphism with Functions

OOP



Abstraction

ABSTRACTION

คลาสนามธรรม (Abstract class) สามารถถือเป็นแบบแผนสำหรับคลาสอื่น ๆ ได้คลาสนี้ช่วยให้คุณสร้างชุดของเมธอดที่ต้องถูกสร้างในคลาสลูกที่สืบทอดมาจากคลาสนามธรรมนี้คลาสที่มีเมธอดนามธรรมหนึ่งเมธอดขึ้นไปจะถูกเรียกว่าคลาสนามธรรม

เมธอดนามธรรม (Abstract method) คือเมธอดที่มีการประกาศแต่ไม่มีการกำหนดเนื้อหาการทำงาน (implementation) เมื่อเรากำลังออกแบบหน่วยฟังก์ชันขนาดใหญ่

เราจะใช้คลาสนามธรรมเมื่อเราต้องการให้มีอินเทอร์เฟซร่วมกันสำหรับการสร้างส่วนประกอบที่แตกต่างกันเราจะใช้คลาสนามธรรม คลาสนามธรรมไม่สามารถสร้างออบเจกต์ได้โดยตรง หมายความว่า你不สามารถสร้างวัตถุจากคลาสนี้ได้ คลาสนี้ถูกใช้เพื่อสืบทอดการทำงานเท่านั้น

ABSTRACTION

```
OOP >  Ex7-Abstract.py > ...
1  from abc import ABC, abstractmethod
2
3  class employee(ABC):
4      def emp_id(self, id, name, age, salary): # Abstraction
5          pass
6
7  class childemployee1(employee):
8      def emp_id(self, id):
9          print("emp_id is 12345")
10
11  emp1 = childemployee1()
12  emp1.emp_id(id)
```

```
emp_id is 12345
```

As you can see in the example, we have imported an abstract method, and the rest of the program has a parent and a derived class. An object is instantiated for the 'childemployee' base class and the functionality of abstract is being used.

ABSTRACTION

OOP >  Ex9-Abs2.py > ...

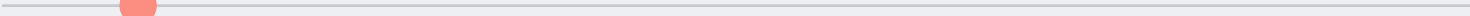
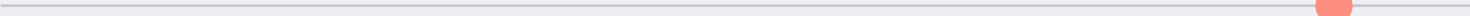
```
1  from abc import ABC, abstractmethod
2  class Absclass(ABC):
3      def print(self,x):
4          print("Passed value: ", x)
5      @abstractmethod
6      def task(self):
7          print("We are inside Absclass task")
8
9  class test_class(Absclass):
10     def task(self):
11         print("We are inside test_class task")
12
13  class example_class(Absclass):
14     def task(self):
15         print("We are inside example_class task")
16
17  #object of test_class created
18  test_obj = test_class()
19  test_obj.task()
20  test_obj.print(100)
21
22  #object of example_class created
23  example_obj = example_class()
24  example_obj.task()
25  example_obj.print(200)
26
27  print("test_obj is instance of Absclass? ", isinstance(test_obj, Absclass))
28  print("example_obj is instance of Absclass? ", isinstance(example_obj, Absclass))
```

```
We are inside test_class task
Passed value:  100
We are inside example_class task
Passed value:  200
test_obj is instance of Absclass?  True
example_obj is instance of Absclass?  True
```

ABSTRACTION

```
OOP > Ex8-OverridingFunction.py > ...
1  class Dog:
2      species = 'mammal'
3
4      def calAge(self,age):
5          print('Dog Age is {}'.format(age*3))
6
7
8  class SomeBread(Dog):
9      pass
10
11 class SomeOtherBread(Dog):
12     species = 'reptile'
13     def calAge(self,age):
14         print('Dog Age is {}'.format(age*4))
15
16 frank = SomeBread()
17 print(frank.species)
18 frank.calAge(5)
19
20 beans = SomeOtherBread()
21 print(beans.species)
22 beans.calAge(5)
```

```
mammal
Dog Age is 15
reptile
Dog Age is 20
```



THANKS

WORD OF THE WISE

